

Railway Reservation System

Code එක class එකෙන් class එකට සරල සිංහලෙන් පැහැදිලි කළ
විස්තරාත්මක වාර්තාව

භාෂාව: Java | **Project type:** Console application

සැකසුවේ: කණ්ඩායම් ඉදිරිපත් කිරීම සඳහා

සටහන: මෙය source code එක පරීක්ෂා කර සකස් කළ report එකකි.
Class, method, data structure, algorithm සහ system flow සියල්ල මෙහි
ඇතුළත් වේ.

1. Project එකේ සරල අදහස

මේ program එක railway ticket reservation system එකකි. User කෙනෙක්ට train බලන්න, ticket book කරන්න, reservation cancel කරන්න, waiting list බලන්න, අවසන් cancellation එක undo කරන්න, passenger සෙවීම, reservation සෙවීම හා sort කිරීම, railway route සහ shortest path බලන්න පුළුවන්.

මෙහි විශේෂත්වය Java ready-made collections පමණක් භාවිත නොකර තමන්ම ලියපු **Linked List, Queue, Stack, Hash Table, BST සහ Graph** වැනි data structures භාවිත කිරීමයි.

System flow එක

```
Main.main()
  ↓
new RailwaySystem() → initRoutes()
  ↓
start() menu loop
  ├── Book → Passenger + Reservation → LinkedList, BST, HashTable, Train
  ├── Cancel → remove from LinkedList/BST → Stack හෝ Waiting Queue passenger
  ├── Search / Sort → reservations වල data භාවිත කරයි
  └── Route → Graph + Dijkstra shortest path
```

Package	Class	වගකීම
railway	Main, RailwaySystem	Program start කිරීම සහ business logic control කිරීම
railway.model	Passenger, Train, Reservation	System එකේ data objects
railway.datastructures	Node, LinkedList, Queue, Stack, HashTable, BST, AVLTree, Graph, HashSetCustom	Data store, organize, search කිරීම
railway.algorithms	Sort, Search	Sorting සහ linear searching
railway.utils	Input	Shared Scanner තබාගැනීම

2. Program එක run වන්නේ කොහොමද?

Main.java

```
RailwaySystem railwaySystem = new RailwaySystem();
railwaySystem.start();
```

Main class එක program එකේ entry point එකයි. Java program එක run වුනාම main method එක මුලින්ම වැඩ කරයි. එය RailwaySystem object එකක් හදලා start() call කරනවා. සැබෑ වැඩ සියල්ල RailwaySystem තුළ.

Model classes — Passenger.java

Passenger කෙනෙකුගේ details තබාගන්න class එක. Fields: passengerId, name, nic, phone. Constructor එක values set කරයි. get... methods private fields safely කියවීමට getters. displayPassenger() console එකේ formatted details print කරයි.

Train.java

Train number, name, source, destination, total seats සහ available seats තබා ගනී. Constructor එකේ availableSeats = totalSeats; මුලදී සියලු seats නිදහස්.

Method	කරන දේ
bookSeat()	Seat තිබේ නම් availableSeats 1කින් අඩු කරයි; negative වෙන්න ඉඩ නොදෙයි.
cancelSeat()	Cancel කරන විට 1 වැඩි කරයි; total seats ට වඩා වැඩි නොවෙයි.
displayTrain()	Train details සහ available seats print කරයි.

Reservation.java

Reservation එකක් යනු **කාගේ ticket එකද + කුමන train එකද + කුමන seat එකද** යන සම්බන්ධයයි. Fields: reservation ID, Passenger object reference, Train object reference සහ seat number. displayReservation() passenger name සහ train name getters හරහා ගෙන print කරයි.

Reservation එක ඇතුළේ passenger details copy කරලා තැහැ; Passenger සහ Train object දෙක refer කරනවා. ඒ නිසා data organized වේ.

3. RailwaySystem.java — program එකේ controller එක

මේ class එක menu එක, validation එක සහ සියලු features එකට සම්බන්ධ කරන මධ්‍යස්ථානයයි.

Field	භාවිතය
Scanner sc	User ගෙන් console input ගනී.
nextPassengerId / nextReservationId	Auto IDs: 1 සහ 1001 සිට වැඩි කරයි.
LinkedList reservations	සියලු active reservations insertion order එකට තබයි.
Queue waitingQueue	Train full වුන අය FIFO waiting list එකට තබයි.
Stack undoStack	අවසන් cancel එක undo කිරීමට.
BST reservationTree	Reservation ID මගින් search කිරීමට.
HashTable passengers	Passenger ID මගින් passenger සෙවීමට.
Graph graph	Stations හා distances තබා shortest path හොයයි.
Train[] trains	Hard-coded trains 3ක array එක.

Constructor සහ menu loop

Constructor එක initRoutes() call කර Graph එක සකස් කරයි. start() තුළ do...while loop එකෙන් menu එක නැවත නැවත පෙන්වයි. User තේරීම switch එකෙන් අදාල method එකට යයි. 11 තෝරන තුරු program එක ක්‍රියා කරයි.

viewTrains() සහ findTrain()

viewTrains() enhanced for-loop එකෙන් trains array එකේ හැම train එකක්ම display කරයි. findTrain(trainNo) linear search එකකි: numbers compare කර match වුන object එක return; නැත්නම් null.

bookTicket() — booking flow

1. Trains display කර user ගෙන් train number ගනී.
2. findTrain null නම් invalid train කියා return වෙයි.
3. Available seats 0 නම් readPassenger() මගින් passenger හදා Queue rear එකට දමයි.
4. Seat තිබේ නම් validSeat() මගින් range සහ duplicate booking check කරයි.
5. හරි නම් addReservation() reservation create කරයි.

addReservation() එකම Reservation object එක Linked List එකටත් BST එකටත් insert කරයි; ඊළඟට train available seats 1කින් අඩු කරයි. මෙය structures දෙක synchronization එකේ තබන වැදගත් තැනයි.

cancelTicket() සහ undoLastCancel()

Cancel කිරීමේදී මුලින් Linked List එකෙන් reservation හොයයි. හමු වී delete වුනොත් BST එකෙන් delete කරයි, train seat count එක වැඩි කරයි.

- Waiting Queue හිස් නම් cancelled reservation එක Stack එකට push කරයි; පසුව undo කළ හැක.
- Queue හිස් නොවේ නම් front passenger dequeue කර එම seat එකට reservation එකක් හදයි. FIFO fairness රැකේ.

undoLastCancel() Stack top reservation එක peek කරයි. Seat එක තවම free නම් pop, Linked List insert, BST insert සහ train.bookSeat() කර restore කරයි.

අනෙක් method	පැහැදිලි කිරීම
passengerMenu()	HashTable එකේ සියලු passengers print කිරීම හෝ ID මගින් search.
searchReservation()	ID නම් BST search; NIC නම් Linked List linear search.
sortReservations()	List එක array එකක් කර selected algorithm එකෙන් sort කර print කරයි. මුල් list එක වෙනස් නොවේ.
routeMenu()	Routes print කිරීම හෝ Dijkstra shortest path run කිරීම.
readPassenger()	ID auto-generate කර input ගෙන HashTable එකට insert කරයි.
validSeat()	Seat range සහ same train/seat duplicate ද බලයි.
readInt()	Integer එකක් ලැබෙන තුරු validation loop.
readLine()	හිස් text accept නොකරයි.
initRoutes()	Stations 7ක් සහ weighted routes 6ක් Graph එකට add කරයි.

4. Linked List සහ Node

Linked List එක array එකක් වගේ contiguous memory අවශ්‍ය නැති list එකකි. එක් node එකක data එකත් ඊළඟ node එකේ reference එකත් තිබේ.

```
head → [Reservation|next] → [Reservation|next] → null
```

Node එකේ Reservation data සහ Node next ඇත. getData/getNext කියවීමට; setData/setNext වෙනස් කිරීමට. LinkedList සහ Stack දෙකම මේ Node class එක reuse කරනවා.

LinkedList method	Code logic	Time
insert	New Node එකක් හදා head නැත්නම් head කරයි. නැත්නම් last node තෙක් ගොස් next එකට link කරයි.	O(n)
display	head සිට null දක්වා node එකින් එක ගොස් print කරයි.	O(n)
search	Reservation ID match වෙන තුරු traverse කරයි.	O(n)
delete	Head නම් head advance කරයි. මැද node නම් previous.next = target.next ලෙස link කරයි.	O(n)
toArray	size ප්‍රමාණ array එකක් හදා nodes වල data copy කරයි.	O(n)
searchByPassengerId / searchByNic	Reservation → Passenger relationship එක භාවිත කර compare කරයි.	O(n)
isSeatBooked	Train number සහ seat number දෙකම match ද බලයි.	O(n)

මෙය reservations display කිරීම, cancel කිරීම, NIC search කිරීම සහ duplicate-seat validation සඳහා භාවිත වෙයි.

5. Queue — waiting list සඳහා FIFO

Queue එකේ නීතිය **FIFO (First In, First Out)** යි. මුලින් waiting list එකට ආ passenger ට මුලින්ම seat එක ලැබිය යුතු නිසා මෙය හොඳ තේරීමකි.

```
front → [A] → [B] → [C] ← rear  
dequeue() removes A; enqueue(D) adds after C
```

QueueNode Passenger එකක් හා next link එකක් තබයි. Queue එකේ front සහ rear pointers දෙකක් ඇත.

- enqueue: rear null නම් front සහ rear දෙකම new node. නැත්නම් old rear.next = new node, rear = new node. O(1).
- dequeue: front data return කර front = front.next. අන්තිම node ඉවත් වුනොත් rear = null. O(1).

- isEmpty: front null ද බලයි; display front සිට traverse කරයි.

6. Stack — Undo සඳහා LIFO

Stack එකේ නීතිය **LIFO (Last In, First Out)** යි. අන්තිමට cancel කළ reservation එක මුලින්ම undo විය යුතු නිසා මෙය සුදුසුයි.

```
top → [අවසාන cancel] → [ඊට පෙර cancel] → null
```

push අලුත් Node එක top ඉදිරියට තබයි; pop top එක return කර top.next වෙත මාරුවෙයි; peek remove නොකර top එක බලයි. තුනම $O(1)$. display top සිට පහළට පෙන්වයි.

7. Hash Table — Passenger ID search

Hash Table එක key එකක් table index එකකට පරිවර්තනය කරන structure එකකි. මෙහි key එක Passenger ID යි. $\text{hash}(\text{key}) = \text{Math.abs}(\text{key}) \% 31$. එම index එක bucket එකක් ලෙස භාවිත වේ.

```
Passenger ID 32 → 32 % 31 = 1 → table[1]
table[1] → Entry(32) → Entry(1) → null (collision chain)
```

IDs දෙකක් එකම index එකට යන **collision** එකක් ආවොත් Entry next හරහා linked chain එකක් (separate chaining) හඳුනවා.

Method	පැහැදිලි කිරීම
insert	Bucket chain එකේ same ID තිබේ නම් update; නැත්නම් new Entry bucket head එකට දමයි.
search	ID hash කර අදාළ bucket chain එකේ compare කරයි.
delete	previous/current references භාවිතයෙන් chain එකෙන් entry remove කරයි.
searchByNic	NIC hash key එක නොවන නිසා buckets සියල්ල traverse කරයි; $O(n)$.
display	31 buckets සියල්ලේ entries print කරයි.

සාමාන්‍යයෙන් ID insert/search $O(1)$ වීමට ඉඩ ඇත; collisions වැඩි නම් $O(n)$ දක්වා යා හැක.

8. BST — Reservation ID search

Binary Search Tree එකේ node එකකට වඩා කුඩා IDs left side; විශාල IDs right side. ඒ නිසා search කරන විට අදාළ branch එක පමණක් තෝරා යා හැක.

```
      1005
     /  \
    1002  1008
search(1008): 1005 → right → 1008
```

- insert: recursive method එකකි. null position එකට new TreeNode; ID අනුව left/right call.
- search: current node ID compare කර left/right යයි.
- delete: child නැත්නම් null; එක් child නම් එම child; children දෙකම නම් right subtree smallest successor ගෙන replace කරයි.
- inorder: left → root → right; IDs ascending order එකට print වේ.

Balanced නම් insert/search/delete $O(\log n)$; නමුත් ascending IDs පමණක් insert කරද්දී මේ සාමාන්‍ය BST එක skewed වෙලා $O(n)$ විය හැක.

9. Graph — railway route සහ shortest path

Graph එකේ station එකක් **vertex** එකකි; station දෙකක් අතර route එක **edge** එකකි. distance weight එක km ගණනයි. adjacencyMatrix[i][j] 0 නම් direct route නැහැ; positive නම් distance ඇත. addEdge දෙපැත්තටම distance set කරන නිසා routes undirected.

```
Colombo -115- Kandy -165- Badulla
|119
Galle -45- Matara
|206
Anuradhapura -198- Jaffna
```

Method	පැහැදිලි කිරීම
indexOf	Station name array එකේ case-insensitive search කර index දෙයි.
displayRoutes	Matrix pairs බලලා direct routes එක වරක් පමණක් print කරයි.
bfs	Java LinkedList queue එකෙන් level-by-level traverse. visited array එක repeat visits වළකයි.
dfs	Recursive call මගින් ගැඹුරට ගොස් පසුව backtrack කරයි.
shortestPath	Dijkstra: අඩුම temporary distance station එක තෝරා neighbors update කරයි. previous array එකෙන් path නැවත හදයි.

මේ fixed small graph එකට adjacency matrix හොඳයි. Dijkstra මෙහි $O(V^2)$ වේ.

10. Sorting algorithms

`sortReservations()` මුලින් `LinkedList` → `Reservation` array conversion කරයි. Sorting array copy එක මත පමණක් නිසා original `LinkedList` insertion order එක වෙනස් නොවේ.

Sort method	කොහෙට භාවිතද?	මූලික logic	Time
Bubble sort	Reservation ID ascending	Neighbor pair compare කර wrong order නම් swap. එක් pass එකකින් largest එක end වෙත යයි.	$O(n^2)$
Selection sort	Passenger name A-Z	Unsorted part එකේ smallest name හොයා current position එකට swap.	$O(n^2)$
Insertion sort	Seat number ascending	Key එක sorted left portion එකේ හරි තැනට shift කර insert.	Worst $O(n^2)$
Merge sort	Train number; method තිබේ, menu එකේ නැත	Array දෙකට බෙදා recursively sort කර merge කරයි.	$O(n \log n)$
Quick sort	Reservation ID descending; menu එකේ නැත	Pivot එකක් තෝරා partition කර recursively sort.	Average $O(n \log n)$

`swap()` helper එක `Reservation` variables දෙක temp variable එකකින් මාරු කරයි. `i` සහ `j` එකම නම් swap එක නවතයි.

11. Search.java, AVLTree.java, HashSetCustom.java සහ Input.java

Search.java

`linearSearch(Reservation[] reservations, int reservationId)` array එක මුල සිට අගට ID compare කරයි. Match එකක් ලැබුණොත් return; නැත්නම් null. Time $O(n)$. දැනට `RailwaySystem` menu එක මේ class එක call නොකරයි; ID search සඳහා BST භාවිත කරයි.

AVLTree.java

AVL Tree එක self-balancing BST එකකි. BST එක IDs ascending order එකෙන් insert වුණොත් tree එක පේළියක් විය හැක. AVL එක height සහ `balanceFactor = left height - right height` භාවිත කර +1, 0, -1 අතර balance තබයි. සීමාව ඉක්මවූ විට `rotateLeft` හෝ `rotateRight` කරයි. Insert, delete, search guaranteed $O(\log n)$ වේ. Code එක තිබුණත් `RailwaySystem` දැනට BST object එක භාවිත කරයි; AVLTree integrate කරලා නැහැ.

HashSetCustom.java

Duplicate NIC නොදැමීමට custom set එකකි. String array එකේ **linear probing** භාවිත කර collision resolve කරයි: home index occupied නම් next slot. contains duplicate check; add valid unique NIC add; array full නම් `resize()` capacity දෙගුණ කර old NIC නැවත insert කරයි. දැනට booking flow එක මෙය use නොකරයි.

Input.java

Public static Scanner එකක් තබන utility class එකකි. RailwaySystem තමාගේ private Scanner එක භාවිත කරන නිසා මේ class එක දැනට unused.

වැදගත්: “Class එක project එකේ තිබීම” සහ “දැනට main menu feature එකකින් call වීම” එකම දෙයක් නොවේ. AVLTree, HashSetCustom, Search සහ Input source code තුළ ඇත; නමුත් current user flow එකට connect වී නැත. Viva එකේ මෙය නිවැරදිව කියන්න.

12. Booking උදාහරණය සහ data structure සම්බන්ධය

User 101 train එකේ seat 5, Nimal සඳහා book කරයි කියලා හිතමු.

1. findTrain(101) → Yal Devi Train object එක ලැබේ.
2. validSeat(train, 5) → seat range හරිද සහ LinkedList එකේ 101/5 නැද්ද බලයි.
3. readPassenger() → Passenger ID 1 ඇති Nimal object එකක් හදලා HashTable එකට insert කරයි.
4. addReservation() → Reservation ID 1001 හදයි.
5. 1001 reservation එක LinkedList last එකටත් BST ID අනුවත් insert වේ.
6. train.bookSeat() → available seats 100 සිට 99 වෙයි.

එය cancel කරන විට List සහ BST දෙකෙන් 1001 ඉවත් වේ; available 99 සිට 100 වෙයි. Waiting queue හිස් නම් 1001 Stack එකට යයි. Undo කළොත් නැවත List/BST එකට insert වී seat count 99 වෙයි.

Requirement	භාවිත කළ structure	ඇයි?
සියලු reservations store/display	Linked List	Dynamic size; nodes හරහා add/delete.
Train full වූ අය	Queue	පළමුව ආ අයට පළමුව seat: FIFO.
Cancel ආපසු ගැනීම	Stack	Latest cancellation එක පළමුව undo: LIFO.
Passenger ID lookup	Hash Table	සාමාන්‍යයෙන් direct-bucket lookup වේගවත්.
Reservation ID lookup	BST	Order අනුව branch කර search වේගවත්.
Stations/routes	Weighted Graph	Connections සහ distance represent කිරීම.
Sorted reports	Array + sorting	Sorting algorithms array මත සරලව run වේ.

13. දැනට තිබෙන සීමා සහ වැඩිදියුණු කළ හැකි තැන්

- Data run-time memory තුළ පමණි; application exit වුනාම reservations නැතිවේ. File/database persistence එකක් එකතු කළ හැක.
- HashSetCustom integrate කළොත් duplicate NIC policy එක enforce කළ හැක.
- Reservation IDs අනුපිළිවෙළින් එන නිසා BST skew විය හැක. AVLTree replace කළොත් $O(\log n)$ search ලැබේ.

- Passenger list HashTable bucket order එකෙන් display වෙයි; ID order එකෙන් නොවෙයි.
- Waiting passenger ට allocation වෙනවිට new reservation ID එකක් ලැබේ. Business rule එක අනුව request ID වෙනම තබාගත හැක.
- Graph BFS/DFS methods තිබුණත් route menu එකේ display/shortest-path පමණි. BFS/DFS options එකතු කළ හැක.
- Sort class එකේ Merge/Quick sort තිබුණත් menu options නැත.

14. කණ්ඩායම 6 දෙනෙකුට presentation බෙදීම

සාමාජිකයා	කියන්න/පෙන්වන්න
1	Introduction, problem statement, Main.java සහ overall flow.
2	Passenger, Train, Reservation models සහ object relationships.
3	RailwaySystem booking/cancel/validation flow live demonstration.
4	Linked List, Queue, Stack: reservation/waiting list/undo example.
5	Hash Table, BST, AVLTree සහ searching concepts.
6	Graph/Dijkstra, sorting algorithms, limitations සහ conclusion.

15. Viva සඳහා කෙටි ප්‍රශ්න-පිළිතුරු

ප්‍රශ්නය	කෙටි පිළිතුර
Queue එකක් ඇයි?	Waiting list එක fair වීමට; පළමුව ඇතුල් වූන passenger ට පළමුව seat.
Stack එකක් ඇයි?	Undo latest cancellation එකෙන් ආරම්භ වන නිසා LIFO.
Duplicate seat check කොහොමද?	LinkedList reservations traverse කර train number සහ seat number compare කරයි.
Hash collision යනු?	Keys දෙකක් එකම table index එකට යාම; මෙහි Entry chain එකෙන් handle කරයි.
BST faster ඇයි?	ID compare කර අදාළ branch එක පමණක් යයි; balanced නම් $O(\log n)$.
Dijkstra එකෙන් මොකක්ද?	Positive distances ඇති graph එකක shortest total-distance path.
AVLTree භාවිත නොකරන්නේද?	Class එක තිබුණත් RailwaySystem දැනට BST භාවිත කරයි.

16. අවසාන නිගමනය

Railway Reservation System එක data structures එකිනෙකට ගැළපෙන real-world case study එකකි. Linked List එක reservations, Queue එක fair waiting list, Stack එක undo, Hash Table එක passenger lookup, BST එක reservation lookup, Graph එක railway network සහ sorting algorithms report display සඳහා යොදාගෙන ඇත. Code flow එකේ ප්‍රධාන කරුණ වන්නේ එක **Reservation object එක structures දෙකක (Linked List + BST) තබා, book/cancel/undo අවස්ථාවල දෙකම update කිරීම යි.**